Chapter 8 — Quantization and Precision Optimization

> *"Quantization is not about making your model less accurate. It is about using exactly as many bits as the task requires — no more, no less. The difference between FP32 and FP8 for LLM inference is like using a crane to move a pencil."* — Production Inference Engineering Principle

---

Chapter Overview

A 70B parameter model in FP32 weighs 280 GB — more than three H100 GPUs can hold. In FP8 it weighs 70 GB and fits on one. In INT4, 35 GB — with room for a generous KV cache. Quantization is the engineering discipline of reducing numeric precision without unacceptable quality loss, and it is arguably the highest-ROI optimization in modern AI infrastructure: a one-time calibration cost that permanently halves or quarters your serving cost.

Sections in this chapter:

- 8.1 Floating Point Formats — The Complete Precision Landscape
- 8.2 Quantization Fundamentals — Math, Granularity, and the Outlier Problem
- 8.3 Post-Training Quantization — GPTQ, AWQ, and SmoothQuant
- 8.4 FP8 and the NVIDIA Transformer Engine
- 8.5 KV Cache Quantization
- 8.6 Quantization-Aware Training and QLoRA
- 8.7 Quantization on AMD — ROCm and CDNA 4
- 8.8 Deployment Decision Framework [r2]
- 8.9 Production Decision Table — AWQ vs GPTQ vs GGUF [r2]
- 8.10 Key Takeaways and Review Questions

8.1 Floating Point Formats — The Complete Precision Landscape

8.1.1 How Floating Point Numbers Work

Every floating point number allocates its bits across three fields: sign, exponent (range), and mantissa (precision). The engineering trade-off in format design is fundamental: **more bits to exponent → wider representable range; more bits to mantissa → finer precision within that**

range. For AI, range matters more than precision — a wide-range, lower- precision format (BF16) outperforms a narrow-range, higher-precision format (FP16) for training, because gradient magnitudes span enormous dynamic ranges.

FLOATING POINT BIT LAYOUTS — COMPLETE REFERENCE:

Format	Sign	Exp	Mantissa	Total	Range	Precision
FP32	1	8	23	32b	±3.4×10^38	~7 decimal digits
BF16	1	8	7	16b	±3.4×10^38 (=FP32!)	~2.4 decimal digits
FP16	1	5	10	16b	±65,504	~3.3 decimal digits
FP8 E4M3	1	4	3	8b	±448	~1.8 decimal digits
FP8 E5M2	1	5	2	8b	±57,344	~1.2 decimal digits
TF32	1	8	10	19b*	±3.4×10^38	~3.3 decimal digits
INT8	–	–	–	8b	-128 to 127	127 distinct values
INT4	–	–	–	4b	-8 to 7	15 distinct values
NVFP4	1	2	1	4b	±6	limited range+prec

*TF32 is stored as 32b in registers; only 19b are used for computation

KEY PROPERTIES FOR AI WORKLOADS:

WHY BF16 REPLACED FP16 FOR TRAINING:
FP16 range: ±65,504. Gradient magnitudes during training routinely exceed this limit → catastrophic overflow and NaN propagation.
BF16 range: ±3.4×10^38 = same as FP32. No overflow for any realistic gradient.
BF16 sacrifice: precision (7 mantissa bits vs FP16's 10 mantissa bits).
For gradients: range matters more than precision → BF16 wins.
For weights: similar conclusion — BF16 range is sufficient.
BF16 is now universal in training (GPT-3, LLaMA, all modern LLMs).

FP8 FORMAT PAIR — WHY TWO FORMATS:
E4M3 (range ±448, higher precision): used for weights and activations.
Forward-pass values are stable; high precision needed to capture detail.
E5M2 (range ±57,344, lower precision): used for gradients.
Gradient magnitudes vary wildly during backprop; wide range needed.
Together: enable full FP8 training without per-layer configuration.

TF32 — THE SILENT A100 TRAINING ACCELERATOR:
TF32 looks like FP32 from the programmer's perspective.
Internally: 8-bit exponent (FP32 range) + 10-bit mantissa (FP16 precision).
Tensor Core computes TF32 GEMM transparently when FP32 input provided.
A100: 312 TFLOPS FP32 vs 19.5 TFLOPS FP64 → TF32 is the default path.
PyTorch 1.7+: torch.backends.cuda.matmul.allow_tf32 = True (default on).
Quality impact: ~1% degradation on precision-sensitive tasks (disable for scientific computing; keep on for ML training — negligible quality loss).

```
  NVFP4 — THE BLACKWELL INNOVATION:
    4-bit float: 1 sign + 2 exponent + 1 mantissa.
    Block scaling: every 16 weights share one FP8 scale factor.
    Range ±6 per block (scale adjusts absolute range).
    B200 peak: 9,000 TFLOPS FP4 = 2× B200 FP8 = 9× H100 BF16.
    First GPU generation where FP4 training achieves accuracy targets (MLPerf
v5.1).
```

8.1.2 Format Sizes and Their Serving Implications

```
PRECISION FORMAT SIZE TABLE — LLaMA-3 70B:

  Format    Bytes/param  Model size  GPUs needed (H100)  Decode tok/s/GPU
  ──────────────────────────────────────────────────────────────────────────
  FP32      4            280 GB      4× H100             Impractical
  BF16      2            140 GB      2× H100 (TP=2)      ~17
  FP8 W8A8  1             70 GB      1× H100             ~34
  INT8 W8A8 1             70 GB      1× H100             ~26
  INT4      0.5           35 GB      1× H100 (fits!)     ~67
  NVFP4*    0.5+scale     ~37 GB     1× B200             ~170
  ──────────────────────────────────────────────────────────────────────────
  * NVFP4 scale overhead: 1 FP8 scale per 16 weights ≈ 6.25% overhead

  DECODE THROUGHPUT FORMULA (memory-bound, batch=1):
    tokens/sec = HBM_bandwidth / model_bytes × 0.70
    H100 BF16: 3350/140 × 0.70 = ~17 tok/s
    H100 INT4: 3350/35  × 0.70 = ~67 tok/s  (4× from halving bytes)

  This linear relationship between bytes and throughput is exact for
  memory-bound decode. Quantization IS bandwidth optimization.
```

8.2 Quantization Fundamentals

8.2.1 The Quantization Formula

```
QUANTIZATION MATH:

  Quantize:   Q(x) = clamp(round(x / scale + zero_point), q_min, q_max)
  Dequantize: x̂   = scale × (Q(x) − zero_point)

  Where:
```

```
    x        = original floating-point value
    Q(x)     = quantized integer value
    scale    = (max(x) - min(x)) / (q_max - q_min)
    zero_point = q_min - round(min(x) / scale)
    q_min/max = integer range bounds (-128 to 127 for INT8)

SYMMETRIC (zero_point = 0, simpler hardware):
  scale = max(|x|) / q_max
  Q(x) = round(x / scale)
  q_max = 127 (INT8) — value -128 wasted for symmetry
  Used by: NVIDIA cuBLAS INT8 GEMM, most weight quantization

ASYMMETRIC (zero_point ≠ 0, better range utilization):
  scale = (max - min) / 255
  zero_point = round(-min / scale)
  Used by: TFLite, ONNX Runtime for activation quantization

QUANTIZATION ERROR ANALYSIS:
  Maximum error per element: |ε|_max ≤ scale / 2

  For INT8 with range [-1.0, +1.0]:
    scale = 2.0 / 255 ≈ 0.00784
    Max error: 0.00392 — ~0.4% relative error for values near max

  CRITICAL PROBLEM — small values suffer most:
    For value x = 0.01 with outlier max = 72.1:
      scale = 72.1 / 127 = 0.568
      Q(0.01) = round(0.01 / 0.568) = round(0.018) = 0
      x̂ = 0 × 0.568 = 0.000
      Error: 100% — the value is completely lost!

    Quantization accuracy loss concentrates on small-magnitude values.
    This is why outliers destroy quantization quality.
```

8.2.2 Granularity — The Most Important Quality Lever

```
QUANTIZATION GRANULARITY COMPARISON:

                Per-Tensor      Per-Channel        Per-Group (g=128)
                1 scale/layer   1 scale/output-row  1 scale/128 weights
  ──────────────────────────────────────────────────────────────────────
  Scale count   1               N_out               N_params / 128
  Scale overhead Negligible     Small               ~3% of weight storage
```

```
  Quality (INT8) Poor           Good                Excellent
  Quality (INT4) Catastrophic   Poor                Acceptable
  Hardware cost  Lowest         Low                 Medium
  Used in        Legacy only    Common INT8         GPTQ, AWQ, GGUF
  ──────────────────────────────────────────────────────────────────────


  WHY GROUP QUANTIZATION WORKS:
    Weight distributions vary significantly WITHIN a single weight matrix.
    One scale per 128 weights captures local variation.
    One scale per entire layer gets dominated by the worst outlier,
    quantizing all other weights to coarser resolution.

  QUALITY COMPARISON (LLaMA-3 70B MMLU):
    BF16 baseline:        80.3%
    INT4 per-tensor:      72.1%  (-8.2%)  ← unacceptable
    INT4 per-group-128:   79.1%  (-1.2%)  ← acceptable
    The difference: 7 MMLU points from granularity alone.
```

8.2.3 The Outlier Problem

```
TRANSFORMER OUTLIER DISTRIBUTION (LLaMA-3 70B, layer 32, typical):

  99.0% of activation values:  [-3.2, +3.2]
  0.1% of values:              [-14.8, +14.8]  ← mild outliers
  0.01% of values:             [-68.4, +72.1]  ← strong outliers!

  Why outliers exist in transformers:
    Attention softmax: certain query-key products consistently spike.
    LayerNorm: some channels develop systematically large representations.
    Residual streams: values accumulate across 80 layers → magnitude growth.

  Outlier impact on INT8 per-tensor quantization:
    scale = max(|x|) / 127 = 72.1 / 127 = 0.568
    Typical value x=0.5: Q(0.5) = round(0.5/0.568) = 1, x̂ = 0.568 (14% error)
    Small value x=0.01: Q(0.01) = round(0.018) = 0, x̂ = 0.000 (100% error!)

  The three algorithmic solutions:
    GPTQ: compensate for quantization error using Hessian correction
    AWQ:  protect outlier-correlated weights by scaling them up first
    SmoothQuant: migrate outlier difficulty from activations to weights
```

8.3 Post-Training Quantization — GPTQ, AWQ, and SmoothQuant

8.3.1 Naive PTQ — The Baseline

```
NAIVE PTQ:
  1. Run model on calibration dataset (128-512 samples)
  2. Record per-tensor min/max for weights and activations
  3. Compute scale = (max - min) / 255
  4. Convert FP16/BF16 weights to INT8/INT4

  Quality (acceptable):
    BERT-base INT8 PTQ:     -0.3% F1
    GPT-2 INT8 PTQ:         -0.8% PPL

  Quality (catastrophic):
    LLaMA-3 70B INT4 PTQ:   -8.5% MMLU  ← Never use naive PTQ for INT4

  Naive PTQ fails for INT4 because:
    With only 15 distinct values, one outlier forces a coarse step size
    that maps most values to 0 or ±1, destroying fine-grained representations.
    GPTQ and AWQ solve this — naive PTQ cannot.
```

8.3.2 GPTQ — Second-Order Weight Quantization

```
GPTQ ALGORITHM (Frantar et al., 2022):

  Core insight: Quantizing weight w_i introduces error ε_i.
  Instead of accepting this error, compensate by adjusting the remaining
  weights in the same row to cancel out ε_i's effect on the output.

  Theoretical basis: OBQ (Optimal Brain Quantization)
    Minimize: ||W X - W_q X||² subject to W_q being INT4
    Solution: process weights column by column, propagating error forward.

  ALGORITHM (one weight matrix W, shape [out × in]):
    Compute: Hessian H = 2 X^T X  (X = calibration activations)
    H[i,i] measures sensitivity of output to weight column i.

    For each column i (in order of increasing sensitivity):
      a. Round W[:,i] to INT4 with current scale
      b. Error: ε = W[:,i] - dequant(INT4(W[:,i]))
      c. Propagate: W[:,i+1:] -= (ε × H[i, i+1:]) / H[i,i]
         (adjust future weights to compensate for this column's error)
```

```
  Result: INT4 weights where errors cancel out across columns
          rather than accumulating independently.

  QUALITY (LLaMA-3 70B, group_size=128):
    BF16 baseline:     80.3% MMLU
    Naive INT4:        71.8% MMLU  (-8.5%)
    GPTQ INT4:         79.0% MMLU  (-1.3%)  ← 7.2% better than naive!

  CALIBRATION COST:
    Time: 4-8 hours on 8× H100 for 70B (Hessian computation per layer)
    Data: 128 calibration samples
    Storage: INT4 weights (35 GB) + scales (~1 GB) for LLaMA-3 70B

  IMPLEMENTATIONS:
    AutoGPTQ (Hugging Face): production-quality, widely used
    llama.cpp: CPU deployment
    ExLlamaV2: optimized INT4 inference with triton kernels
    TRT-LLM: native GPTQ support
```

8.3.3 AWQ — Activation-Aware Weight Quantization

```
AWQ (Lin et al., 2023):

  Core insight: ~1% of weight channels correspond to "salient" activation
  channels (large activation magnitudes). These have outsized impact on quality.
  Protect the salient weights; quantize everything else aggressively.

  AWQ ALGORITHM:

  Step 1: Find salient channels
    Run calibration data → measure per-channel activation magnitude:
      a_i = mean(|X[:,i]|) for each input channel i
    Top 1% by magnitude = salient channels

  Step 2: Scale up salient weights (make them INT4-safe)
    For salient channel i with scale factor s_i > 1:
      W[:,i] *= s_i       (scale up weight — now uses more INT4 range)
      X[:,i] /= s_i       (inverse on activation — net math unchanged!)

    Net effect on output: W × X = (W×s) × (X/s) = unchanged.
    Net effect on quantization: salient weights now quantize with less error.
```

```
  Step 3: Quantize all weights to INT4 (group_size=128)
    Salient channels already protected → whole matrix quantizes better.

  Step 4: Store per-channel scales s_i alongside INT4 weights.
    At inference: apply s_i to activations before GEMM (fused kernel).

  QUALITY (LLaMA-3 70B W4A16):
    BF16:              80.3% MMLU
    GPTQ INT4:         79.0% MMLU  (-1.3%)
    AWQ INT4:          79.5% MMLU  (-0.8%)  ← better than GPTQ by 0.5%

  AWQ ADVANTAGES over GPTQ:
    2-3× faster calibration (no Hessian computation per layer)
    Better quality on models with pronounced outliers (Mixtral, DeepSeek)
    Similar performance on code benchmarks; AWQ wins on knowledge tasks

  IMPLEMENTATIONS:
    AutoAWQ (Hugging Face): most popular, vLLM native support
    TRT-LLM: AWQ kernel support
    llama.cpp: Q4_K_M is an AWQ-variant format
```

8.3.4 SmoothQuant — Enabling W8A8

```
SMOOTHQUANT (Xiao et al., 2022):

  Problem: Activations have outliers (hard INT8).
           Weights are smooth (easy INT8).
           Naive W8A8 fails because activation outliers destroy scale.

  Solution: MIGRATE difficulty from activations to weights.

  Mathematical equivalence:
    Y = X × W = (X × diag(s)^{-1}) × (diag(s) × W)
         ↑ smoothed activations     ↑ scaled weights
    Output Y is unchanged (diag(s) and its inverse cancel exactly).
    But: smoothed activations have smaller max → easier to quantize.

  Scale selection:
    s_i = max(|X[:,i]|)^α / max(|W[i,:]|)^(1-α)
    α = 0.5: geometric mean, balances activation/weight difficulty
    α → 1: more migration to weights (activations become easy, weights harder)
    α = 0.5 works well for most transformer layers.
```

```
  QUALITY (LLaMA-3 70B W8A8):
    BF16:                    80.3% MMLU
    Naive W8A8 INT8:         75.1% MMLU  (-5.2%)  ← fails without smooth
    SmoothQuant W8A8:        79.6% MMLU  (-0.7%)  ← acceptable!

  THROUGHPUT: ~1.5× vs BF16 (INT8 Tensor Cores on H100/A100)

  WHEN TO USE vs FP8:
    SmoothQuant W8A8: best for A100 (no FP8 hardware).
    FP8 W8A8 (Transformer Engine): better quality AND throughput on H100.
    On H100: prefer FP8. On A100 or AMD: SmoothQuant is primary W8A8 path.
```

8.4 FP8 and the NVIDIA Transformer Engine

8.4.1 The Transformer Engine Mechanism

```
TRANSFORMER ENGINE (TE) ARCHITECTURE:

  Goal: Automatically use FP8 for GEMMs (2× throughput vs BF16)
  while maintaining BF16 quality across the full forward and backward pass.

  WITHOUT TE (BF16):
    QKV GEMM:  BF16 × BF16 → BF16   (989 TFLOPS peak)
    FFN GEMM:  BF16 × BF16 → BF16   (989 TFLOPS peak)

  WITH TE (FP8):
    QKV GEMM:  FP8 E4M3 × FP8 E4M3 → FP32 (accumulated)  (~1,979 TFLOPS!)
    FFN GEMM:  FP8 E4M3 × FP8 E4M3 → FP32                (~1,979 TFLOPS!)
    Norms/activations: BF16 unchanged
    Gradients: FP8 E5M2 (wider range for gradient variance)

  THE CRITICAL MECHANISM — PER-TENSOR DYNAMIC SCALING:

    Step 1: Monitor
      TE tracks amax_history = rolling max of |activation| over last N steps.
      This predicts whether FP8's ±448 range will be sufficient next step.

    Step 2: Compute scale
      scale = 448 / amax_history
      (fits current max into FP8 range)
```

```
    Step 3: Quantize
      X_fp8 = clamp(X_bf16 × scale, -448, +448)  →  FP8 E4M3

    Step 4: GEMM in FP8
      Y_fp32 = X_fp8 × W_fp8   (Tensor Core FP8 mode, ~1,979 TFLOPS)

    Step 5: Descale
      Y_bf16 = Y_fp32 / (scale_X × scale_W)  →  back to BF16

    THIS ENTIRE FLOW IS AUTOMATIC. User changes one line of code.

  CODE — TE INFERENCE:
    import transformer_engine.pytorch as te

    # Replace nn.Linear with te.Linear
    self.qkv = te.Linear(d_model, 3 * d_model, bias=False)

    # Enable FP8 context
    with te.fp8_autocast(enabled=True):
        output = model(input)

    # That's it. No other changes required.

  QUALITY RESULTS (LLaMA-3 70B):
    BF16 baseline:    MMLU 80.3%, HellaSwag 87.1%, TruthfulQA 44.7%
    FP8 W8A8 TE:      MMLU 80.1%, HellaSwag 86.9%, TruthfulQA 44.5%
    Delta:            -0.2%        -0.2%             -0.2%
    → Effectively lossless for all production benchmarks.
```

8.4.2 FP8 Training — The Full Mixed-Precision Stack

```
FP8 TRAINING MEMORY (LLaMA-3 70B with TE):

  FP8 weights (E4M3):         70B × 1B  =  70 GB
  FP32 master weights:        70B × 4B  = 280 GB
  FP8 gradients (E5M2):       70B × 1B  =  70 GB  (vs 140 GB BF16!)
  FP32 Adam m + v:            70B × 8B  = 560 GB
  ──────────────────────────────────────────────────────────────
  Total training state:       70B × 14B = 980 GB  (vs 1,120 GB BF16)
  Savings vs BF16 training:   140 GB (12.5%)

  TRAINING THROUGHPUT GAIN:
    FP8 GEMM: ~1,979 TFLOPS vs BF16 ~989 TFLOPS = 2× raw throughput
```

```
    Practical training speedup (system-level): ~1.5-1.9× depending on
    communication bottlenecks and non-GEMM operations.

  FP8 TRT-LLM BUILD (inference deployment):
    trtllm-build \
      --checkpoint_dir ./llama3-70b-hf \
      --output_dir ./llama3-70b-trt \
      --dtype bfloat16 \
      --quantization fp8 \
      --calib_dataset cnn_dailymail \
      --calib_batches 512

    Build time: 10-30 min (calibration + TRT engine compilation)
    Result: .engine file with FP8 weights, optimal CUDA Graphs
    Throughput: ~2× BF16 for decode, ~2× for prefill
    No runtime calibration overhead — scales baked in at build time.
```

8.5 KV Cache Quantization

The KV cache is the second-largest memory consumer in inference after the model weights. Quantizing it independently from weights is a high-ROI technique.

8.5.1 Why KV Cache Can Be Quantized Independently

```
KV CACHE QUANTIZATION — MOTIVATION:

  For LLaMA-3 70B (GQA-8, BF16) at S=4096:
    KV cache: 1.34 GB per sequence
    At batch=32, H100 (80 GB total):
      Model weights: 70 GB (FP8)
      KV for 32 seqs: 32 × 1.34 GB = 42.9 GB
      Available HBM: 80 - 70 = 10 GB → only 7 sequences fit!

  INT8 KV cache:
    KV per sequence: 0.67 GB (halved)
    At 10 GB available: 14 sequences fit (2× more)

  FP8 KV cache:
    KV per sequence: 0.67 GB (same as INT8 — both 1 byte/element)
    Better quality than INT8 for same compression

  WHY KV QUANTIZATION IS RELATIVELY SAFE:
```

```
    KV values are attention keys and values — they participate in dot products.
    The softmax in attention is naturally robust to small KV errors.
    Unlike weight quantization (which accumulates errors across layers),
    KV quantization error affects only the current attention step.
```

8.5.2 KV Cache Quantization Techniques

```
KV CACHE QUANTIZATION OPTIONS:

INT8 KV CACHE:
  Per-token scale: one scale per token (not per tensor or per channel).
  Q(KV[t]) = round(KV[t] / scale_t) for each token position t.
  scale_t = max(|KV[t]|) / 127 (symmetric INT8)

  Quality: -0.3 to -0.8% MMLU (less than W8A8 activation quant)
  Memory: 2× reduction vs BF16 KV
  Implementation: vLLM --kv-cache-dtype fp8_e5m2 (supported)
                  TRT-LLM: native INT8 KV

FP8 KV CACHE:
  Use E5M2 format (wider range than E4M3 for KV values).
  Typically dynamic per-token quantization.
  Quality: -0.2% MMLU (slightly better than INT8)
  Memory: 2× reduction vs BF16 KV (same as INT8)
  Hardware: H100, H200, B200

COMBINED GQA + FP8 KV — THE HIGH-ROI STACK:

  KV bytes per token per layer:
    BF16 MHA (H_kv=64, d=128):  2 × 64 × 128 × 2B = 32,768 bytes/tok
    BF16 GQA (H_kv=8, d=128):   2 × 8 × 128 × 2B  = 4,096 bytes/tok
    FP8 GQA  (H_kv=8, d=128):   2 × 8 × 128 × 1B  = 2,048 bytes/tok

  Comparison at S=4096, L=80:
    BF16 MHA:  32,768 × 4096 × 80 = 10.7 GB per sequence
    FP8 GQA:    2,048 × 4096 × 80 = 0.67 GB per sequence
    Reduction: 16.0× fewer KV bytes!

  CONCURRENT USER CAPACITY (H100, 70B FP8 weights, 10 GB KV budget):
    BF16 MHA: 10 GB / 10.7 GB = 0 (doesn't fit, need larger GPU!)
    BF16 GQA: 10 GB / 1.34 GB = 7 concurrent sequences
    FP8 GQA:  10 GB / 0.67 GB = 14 concurrent sequences  ← 2× more
```

```
  REAL NUMBERS (H100 80 GB, 70 GB FP8 weights, 10 GB KV budget, S=4096):
    BF16 MHA:   10 GB / (32,768 × 4096 × 80 / 1e9) = 80 max requests
    FP8 GQA:    10 GB / (2,048 × 4096 × 80 / 1e9)  = 1,220 max requests
    Improvement: 15.3× more concurrent users per GPU!

  This combination (GQA + FP8 KV) is one of the highest-ROI inference
  optimizations available — two architectural decisions that multiply together.
```

8.6 Quantization-Aware Training and QLoRA

8.6.1 QAT — Training Through Quantization Error

```
QUANTIZATION-AWARE TRAINING (QAT):

  Core idea: Simulate quantization error during training so the model
  can adapt its weights to work around the precision loss.

  FAKE QUANTIZATION (forward pass):
    def fake_quantize(x, bits=4, scale=...):
        q = round(x / scale) * scale   # round to nearest INT4 grid
        q = clamp(q, min_val, max_val) # clip to representable range
        return q                       # float, but with INT4 error

    Model sees the error during forward → learns to compensate.

  STRAIGHT-THROUGH ESTIMATOR (backward pass):
    Problem: round() has zero gradient everywhere → no learning signal.
    STE solution: d(fake_quant)/dx = 1.0 if |x| ≤ max_val, else 0.0
    Approximates gradient as identity within the valid range.
    This is the standard gradient trick for discrete quantization.

  QUALITY (LLaMA-3 7B, INT4):
    BF16 baseline:   MMLU 65.4%
    GPTQ INT4 PTQ:   MMLU 63.8%  (-1.6%)
    AWQ INT4 PTQ:    MMLU 64.1%  (-1.3%)
    QAT INT4:        MMLU 64.9%  (-0.5%)  ← best quality for INT4

  QAT COST:
    Compute: ~10-20% of original pretraining cost (not cheap!)
    When worth it: high-volume production INT4 serving (ROI from cost/token)
    When not worth it: FP8 (free with TE), one-off or low-volume deployment
```

8.6.2 QLoRA — Fine-Tuning 70B on One GPU

```
QLORA (Dettmers et al., 2023):

  QLoRA = NF4-quantized base model + BF16 LoRA adapters
  Enables fine-tuning 65B+ models on a single 48 GB GPU.

  NF4 FORMAT (NormalFloat 4-bit):
    Optimized for normally-distributed weights.
    Instead of evenly-spaced INT4 grid, uses a grid where more values
    cluster around 0 (matching the near-normal distribution of LLM weights).
    Better quality than uniform INT4 for the same bit count.

  QLORA ARCHITECTURE:
    Base weights:    NF4 (4-bit, loaded and kept in 4-bit)
    Computation:     On-the-fly dequantize NF4 → BF16, then GEMM
    LoRA adapters:   BF16 (full precision; only these are updated)
    Double quant:    Also quantize the scale factors (saves 0.37 bits/param)

  TRAINING LOOP:
    1. Load model in 4-bit (NF4) via bitsandbytes
    2. Add LoRA adapters (r=64 for high quality)
    3. Forward: NF4 → BF16 dequant → GEMM → LoRA delta
    4. Backward: only LoRA A,B gradients computed
    5. Update: only LoRA parameters updated

  MEMORY (LLaMA-3 70B QLoRA):
    NF4 weights:          35 GB
    BF16 LoRA (r=64):      0.5 GB
    Activations:           ~8 GB
    Adam on LoRA only:     1 GB
    TOTAL:                ~44.5 GB  → fits on 1× A6000 48 GB!

  QUALITY:
    Full fine-tune BF16 (8× H100):  95% quality baseline
    QLoRA (1× 48 GB GPU):           93-94% of full fine-tune
    Gap: 1-2% — acceptable for most instruction-following tasks
    Use case: fine-tuning when full parameter fine-tuning is infeasible

  PYTHON SETUP:
    from transformers import BitsAndBytesConfig
    from peft import LoraConfig, get_peft_model
```

```
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,   # quantize the scales too
    )
    lora_config = LoraConfig(
        r=64,
        lora_alpha=16,
        target_modules=["q_proj", "v_proj", "k_proj", "o_proj",
                        "gate_proj", "up_proj", "down_proj"],
    )
    model = AutoModelForCausalLM.from_pretrained(
        "meta-llama/Llama-3-70b-hf",
        quantization_config=bnb_config
    )
    model = get_peft_model(model, lora_config)
    # Standard Trainer loop follows...
```

8.7 Quantization on AMD — ROCm and CDNA 4

```
AMD QUANTIZATION SUPPORT (MI300X / MI350X):

  FP8 SUPPORT:
    ROCm 6.x: FP8 E4M3 and E5M2 support in HIP/ROCm kernels.
    Matrix Core FP8: supported on CDNA 3 (MI300X) for GEMM.
    Equivalent to NVIDIA Transformer Engine in concept;
    AMD's implementation via CK (Composable Kernel) library.
    vLLM ROCm: FP8 support added in vLLM 0.6+ for MI300X.
    Quality: comparable to NVIDIA FP8 at same group sizes.

  INT4 / AWQ:
    ROCm 6.x: AutoAWQ supports MI300X via HIP backend.
    Throughput gain: similar to NVIDIA (bandwidth reduction dominates decode).
    Library support: flash-attention ROCm, vLLM ROCm.

  MI300X QUANTIZATION ADVANTAGE:
    BF16 LLaMA-3 70B: fits on single MI300X (192 GB), no TP needed.
    FP8 LLaMA-3 70B (70 GB): fits with 122 GB remaining for KV cache!
    At FP8 + FP8 KV + S=128K: 70 GB weights + 42 GB KV = 112 GB → fits!
    H100 with same config (80 GB): 70 + 42 = 112 GB — requires TP=2.
```

```
    MI300X memory advantage persists even at FP8 for long-context serving.

  CDNA 4 (MI350X) — FP4 SUPPORT: [ANNOUNCED]
    MI350X adds FP4/FP6 Matrix Core support.
    AMD's answer to NVIDIA NVFP4 (B200 FP4).
    FP4 inference on MI350X: ~2.8× MI300X compute at same TDP.
    8.0 TB/s HBM3e bandwidth: decode throughput ~5× MI300X INT4.
    Software stack: ROCm 7.x expected to include FP4 kernels.
```

8.8 Deployment Decision Framework [r2]

8.8.1 Step-by-Step Selection Process

```
QUANTIZATION DEPLOYMENT DECISION FRAMEWORK (5 STEPS):

STEP 0 — CHECK IF QUANTIZATION IS NEEDED:
  Model fits on target GPU(s) at BF16? Calculate: N_params × 2B
  LLaMA-3 70B: 140 GB → need TP=2 on H100 or 1× H200/MI300X
  If fitting requires TP=2+ at BF16: quantization likely worth evaluating.
  If fitting is comfortable: measure throughput delta before deciding.

STEP 1 — HARDWARE PLATFORM:
  NVIDIA H100/H200:  → FP8 W8A8 (TE, native hardware, best quality)
  NVIDIA A100:        → SmoothQuant W8A8 INT8 (no FP8 hardware)
  NVIDIA B200:        → NVFP4 W4A4 (native hardware, peak throughput)
  AMD MI300X:         → FP8 (ROCm 6.x) or AWQ W4A16
  AMD MI350X:         → FP4/FP6 CDNA 4 (when ROCm support matures)
  CPU / Apple Silicon: → GGUF Q4_K_M (llama.cpp optimized)

STEP 2 — QUALITY CONSTRAINT:
  Require < 0.3% benchmark delta:  FP8 W8A8 (TE) or W8A8 SmoothQuant
  Require < 1.0% delta (typical):  AWQ INT4 W4A16 or GPTQ INT4 W4A16
  Require < 2.0% delta:            GPTQ W4 group=64, QLoRA NF4
  Edge use, > 2.0% acceptable:     GGUF Q2-Q3 formats

STEP 3 — MEMORY CONSTRAINT:
  Need 2× fewer GPUs (vs BF16): FP8 W8A8 (same parameter count, half bytes)
  Need 4× fewer GPUs: AWQ/GPTQ INT4 (quarter bytes per parameter)
  Extreme constraint (laptop): INT4 GGUF (0.5 bytes/param)

STEP 4 — CALIBRATION BUDGET:
```

```
  Zero calibration time: FP8 (dynamic scaling, no offline calibration)
  2-3 hours (reasonable): AWQ (activation stats only, fast)
  4-8 hours (acceptable): GPTQ (Hessian computation, more accurate)
  1-2 days: QAT (full retraining required)

STEP 5 — VALIDATE BEFORE PRODUCTION:
  Run benchmark suite: WikiText PPL, MMLU, task-specific eval
  Target: benchmark delta within accepted threshold (Step 2)
  Check: does the quantized model maintain quality on YOUR specific task?
  If not: try per-group granularity if per-channel was used, or switch format.
```

8.8.2 Economic Impact at Scale

```
QUANTIZATION ANNUAL SAVINGS — 10M REQUESTS/DAY (LLaMA-3 70B, H100):

  Assume: 500 output tokens/request
  Scale:  10M × 500 = 5B tokens/day = 1.825T tokens/year

  Config              Tok/s/GPU  GPUs needed  Cost/hr/GPU  $/1M tokens
  ──────────────────────────────────────────────────────────────────────
  BF16 (2×H100 TP=2)  1,530      ~2           $3.32       $0.603
  FP8 (1×H100)        2,200      ~1           $1.66       $0.210
  AWQ INT4 + FP8 KV   2,543      ~1           $1.66       $0.169
  ──────────────────────────────────────────────────────────────────────

  Annual cost (BF16):               1,825,000 × $0.603 = $1,100,475
  Annual cost (AWQ INT4 + FP8 KV):  1,825,000 × $0.169 =   $308,425
  Annual savings:                                          $792,050

  At hyperscaler scale (1T tokens/day): $162M annual savings.
  This is why quantization is not optional at production scale.
  The one-time calibration cost (4-8 hours) pays back in less than one day.
```

8.9 Production Decision Table — AWQ vs GPTQ vs GGUF [r2]

8.9.1 Format Selection by Use Case

```
USE CASE → FORMAT DECISION TABLE:

  USE CASE                                RECOMMENDED    SECOND CHOICE  AVOID
  ──────────────────────────────────────────────────────────────────────────
```

```
Production NVIDIA serving (H100/A100) AWQ INT4       FP8 W8A8       GGUF
Max throughput (H100, BF16 quality)   FP8 W8A8       AWQ INT4       GPTQ
Local CPU inference (laptop)          GGUF Q4_K_M    GGUF Q5_K_M   AWQ
Edge (Raspberry Pi, mobile)           GGUF Q2_K      GGUF Q3_K_M   GPTQ
Fine-tuned model (LoRA adapters)      AWQ INT4       QLoRA NF4      GGUF
Long context (128K+), limited HBM     AWQ INT4       FP8 W8A8       BF16
Research / exact quality comparison   BF16           FP8 W8A8       INT4
Models with strong outliers           AWQ            SmoothQuant    GPTQ
MoE models (Mixtral, DeepSeek)        AWQ INT4       FP8 W8A8       GPTQ
Apple Silicon (M-series)              GGUF Q4_K_M    GGUF Q6_K      AWQ
AMD MI300X production                 FP8 (ROCm 6+)  AWQ INT4       GGUF
B200/GB300 maximum performance        NVFP4 W4A8     FP8 W8A8       INT4
──────────────────────────────────────────────────────────────────────────

TIER RULE:
  NVIDIA H100/A100 production: default to AWQ INT4.
  Quality delta unacceptable (< 0.5% MMLU): use FP8.
  Non-NVIDIA or CPU deployment: use GGUF.
```

8.9.2 Speed and Quality Reference Tables

```
THROUGHPUT COMPARISON (LLaMA-3 70B, H100 SXM5, batch=32, decode): [ESTIMATED]

  Format        Weights  Acts   Tok/s   vs BF16  Memory   Supported HW
  ──────────────────────────────────────────────────────────────────────
  BF16          BF16     BF16   1,530   1.00×    140 GB   Any (req 2×H100)
  FP8 W8A8      FP8      FP8    2,200   1.44×     70 GB   H100, H200, B200
  INT8 SQ       INT8     INT8   1,650   1.08×     70 GB   A100, H100
  AWQ W4A16     INT4     BF16   2,543   1.66×     36 GB   H100, A100
  GPTQ W4A16    INT4     BF16   2,480   1.62×     36 GB   H100, A100
  W4A8 (AWQ+)   INT4     FP8    2,800   1.83×     36 GB   H100 (FP8 acts)
  GGUF Q4_K_M   ~4.5b    BF16   ~820    0.54×    ~40 GB   CPU (H100 possible)
  NVFP4         FP4      FP8    6,800   4.44×    ~18 GB   B200 only
  ──────────────────────────────────────────────────────────────────────

  CALIBRATION TIME (one-time cost):
    FP8 (TE dynamic):   0 hours   (no offline calibration)
    AWQ:                2-3 hours (activation stats, 8×H100)
    GPTQ:               4-8 hours (Hessian computation, 8×H100)
    SmoothQuant:        1-2 hours (scale optimization)
    GGUF:               15-45 min (post-process, CPU)
    QAT:                Days      (full training cycle)
```

```
QUALITY COMPARISON (LLaMA-3 70B):

  Format            MMLU     HumanEval  MT-Bench  WikiText PPL  Δ MMLU
  ──────────────────────────────────────────────────────────────────────
  BF16 baseline     80.3%    69.2%      8.91      3.41          —
  FP8 W8A8          80.1%    68.9%      8.89      3.43          -0.2%
  AWQ W4A16 g=128   79.5%    68.1%      8.82      3.58          -0.8%
  GPTQ W4A16 g=128  79.0%    67.4%      8.76      3.64          -1.3%
  W4A8 AWQ          79.3%    67.8%      8.80      3.61          -1.0%
  GGUF Q5_K_M       79.4%    68.0%      8.83      3.57          -0.9%
  GGUF Q4_K_M       78.8%    66.9%      8.71      3.72          -1.5%
  GGUF Q3_K_M       77.1%    64.2%      8.52      4.11          -3.2%
  GGUF Q2_K         73.1%    58.1%      7.94      6.23          -7.2%
  Naive INT4        71.8%    55.3%      7.61      6.77          -8.5%
  ──────────────────────────────────────────────────────────────────────

  KEY OBSERVATIONS:
    1. AWQ consistently beats GPTQ by ~0.5% MMLU for INT4.
       On code (HumanEval), GPTQ sometimes catches up.
       For knowledge-intensive tasks: prefer AWQ.
    2. GGUF Q5_K_M matches AWQ quality at slightly higher bit count.
       GGUF Q4_K_M is a fair alternative to AWQ on CPU.
    3. FP8: essentially lossless for all metrics. Use whenever hardware supports.
    4. Naive INT4: never use in production. GPTQ/AWQ required for acceptable INT4.
```

8.9.3 Quantization Evaluation Methodology

```
BENCHMARK SUITE FOR QUANTIZATION VALIDATION:

TIER 1 — Language Modeling (Perplexity):
  Dataset: WikiText-2, first 256 tokens per sample
  Metric: PPL = exp(avg negative log-likelihood)
  Target: PPL increase < 1.0 point vs BF16 baseline
  Why: PPL is sensitive to any precision loss; catches subtle regressions.

TIER 2 — Broad Capability:
  MMLU (57 subjects, 5-shot):   knowledge retention
  HellaSwag:                     commonsense reasoning
  ARC-Challenge:                 science QA
  TruthfulQA:                   factual accuracy (sensitive to quantization)
  Target: < quality_threshold (see Step 2 of framework)

TIER 3 — Task-Specific (ALWAYS run before production deployment):
```

```
  Math (GSM8K, MATH): quantization may hurt arithmetic step-by-step
  Code (HumanEval, MBPP): quantization often hurts code specifically
  Your target task: if serving a specific application, eval on real examples
  Why: aggregate benchmarks miss task-specific regressions.

ANTI-PATTERN: Evaluating on MMLU only and shipping to production.
  A model can lose 15% on coding tasks while MMLU shows -0.5%.
  Always include your deployment task in the eval suite.

MINIMAL EVALUATION SCRIPT:
  # Run after calibration, before deploying
  lm-eval --model vllm \
          --model_args pretrained=./model_quantized \
          --tasks mmlu,hellaswag,truthfulqa_mc2 \
          --batch_size 32 \
          --output_path ./quantization_results.json
```

8.10 Key Takeaways and Review Questions

```
QUANTIZATION NUMBERS TO MEMORIZE COLD:

  Format sizes:
    FP32:   4 B/param → 70B = 280 GB
    BF16:   2 B/param → 70B = 140 GB (standard training baseline)
    FP8:    1 B/param → 70B =  70 GB
    INT4:   0.5 B/param → 70B = 35 GB

  Throughput vs BF16 decode (batch=1, H100, memory-bound):
    FP8 W8A8:   2× (halved bytes)
    INT4 W4A16: 4× (quartered bytes)
    NVFP4:      8× vs BF16 (B200 only)

  Quality loss (LLaMA-3 70B MMLU, approximate):
    FP8 W8A8:     -0.2%  (effectively lossless)
    AWQ INT4:     -0.8%  (acceptable production)
    GPTQ INT4:    -1.3%  (acceptable production)
    Naive INT4:   -8.5%  (NEVER use in production)

  KV cache quantization:
    INT8/FP8 KV: 2× more sequences per GPU
    GQA + FP8 KV vs MHA BF16: 16× more concurrent sequences
```

```
KEY PRINCIPLES:

  1. QUANTIZATION = BANDWIDTH OPTIMIZATION FOR DECODE:
     At batch=1, decode throughput ∝ 1 / bytes_per_param (exactly).
     Halving bytes doubles decode throughput. Always.
     INT4 (0.5 B/param) is not "4-bit magic" — it is a 4× bandwidth reduction.

  2. FP8 ON H100 IS THE DEFAULT STARTING POINT:
     Zero calibration cost. Zero code changes with TE. -0.2% quality.
     If deploying on H100/H200/B200: FP8 first, then evaluate if more needed.

  3. AWQ > GPTQ FOR MOST WORKLOADS:
     Faster calibration (2-3h vs 4-8h), similar or better quality.
     Exception: some code and math tasks where Hessian correction helps.
     Exception: GPTQ has broader ecosystem support for some edge runtimes.

  4. GRANULARITY IS CRITICAL FOR INT4:
     Per-tensor INT4: -8.5% MMLU (unacceptable).
     Per-group-128 INT4: -1.3% MMLU (acceptable).
     Never use per-tensor quantization for INT4. Always use group_size=128.

  5. KV CACHE QUANTIZATION IS INDEPENDENT AND COMPLEMENTARY:
     Quantize weights to FP8 → halve weight bytes.
     Quantize KV to FP8 → halve KV bytes.
     Both together → more concurrent users, not faster per-token time.

  6. ALWAYS EVALUATE ON YOUR DEPLOYMENT TASK:
     MMLU is not your production task. Run your eval before shipping.
     A 0.5% MMLU delta may hide a 15% regression on your specific use case.
```

Review Questions

Difficulty: Foundational

Question 1 (Numeric Format Properties): (a) A LLaMA-3 70B model requires gradient computation during pretraining. Explain why BF16 replaced FP16 as the training standard, referencing the specific bit-field that makes BF16 safer. (b) FP8 has two variants: E4M3 and E5M2. Explain which variant is used for weights/activations vs gradients, and why the choice differs. (c) TF32 is described as "invisible" to PyTorch users. What does this mean, and how does it achieve 16× speedup over FP64 on A100?

Expected answer: (a) FP16 has 5 exponent bits → range ±65,504. Gradient magnitudes during training routinely exceed this (especially early training and in large models) → overflow → NaN

propagation → training divergence. BF16 has 8 exponent bits (same as FP32) → range ±3.4×10^38. The identical exponent field gives BF16 the same dynamic range as FP32, eliminating overflow. The sacrifice: BF16 has 7 mantissa bits vs FP16's 10, giving lower precision — but for gradients, range matters more than precision. (b) E4M3 (4 exponent, 3 mantissa): range ±448, higher precision. Used for weights and activations in forward pass where values are relatively stable and precision matters. E5M2 (5 exponent, 2 mantissa): range ±57,344, lower precision but wider range. Used for gradients — backprop gradient magnitudes can spike dramatically (especially near initialization or at large learning rates), so wider range is critical to prevent clipping. (c) TF32 is a compute format, not a storage format. From the user's perspective, they write FP32 code (torch.float32). The hardware transparently casts FP32 to TF32 (truncating mantissa from 23 to 10 bits) inside the Tensor Core multiply, then accumulates in FP32. The user reads FP32 results. PyTorch 1.7+ enables this by default via torch.backends.cuda.matmul.allow_tf32=True. 16× speedup: A100 delivers 312 TFLOPS TF32 vs 19.5 TFLOPS FP64 = 16× — all from using TF32 GEMM paths instead of FP64 paths.

Difficulty: Intermediate

Question 2 (Outlier Impact and AWQ): A LLaMA-3 70B model has activation outliers with values up to ±72.1. You apply symmetric INT8 per-tensor quantization. (a) What scale factor results? (b) What is the quantization error for a typical small activation x=0.02? (c) What fraction of information is lost for x=0.02? (d) How does AWQ specifically address this problem, and what is the net mathematical impact on the output Y = X × W?

Expected answer: (a) scale = max(|x|) / 127 = 72.1 / 127 = 0.5677. (b) Q(0.02) = round(0.02 / 0.5677) = round(0.0352) = 0. $\hat{x}$ = 0 × 0.5677 = 0.000. Error = 0.000 − 0.020 = −0.020. (c) Fraction lost = |error| / |x| = 0.020 / 0.020 = 100%. The value is completely destroyed. (d) AWQ identifies that x=0.02 lies in a high-magnitude activation channel (correlated with the outlier at 72.1). It computes scale s_i for this channel. Then scales the corresponding weight column: W[:,i] *= s_i, and the activation: X[:,i] /= s_i. Net output: Y = X × W = (X/s) × (Ws) = X × W (unchanged). But now: the weight column W[:,i] × s_i has larger magnitude → uses more of INT4's range → quantizes more accurately. The activation X[:,i] / s_i is smaller → less of an outlier → scale computed for all activations is smaller → step size smaller → small values quantize more precisely. The information at x=0.02 is no longer destroyed because the overall scale is no longer dominated by the 72.1 outlier after AWQ scaling.

Difficulty: Principal Level

Question 3 (Deployment Decision): Your team needs to serve LLaMA-3 70B on H100 SXM5 at 50M tokens/day with P99 TTFT < 400ms and < 1.0% MMLU quality delta from BF16. You have exactly 8 H100 SXM5 GPUs. Design the complete quantization strategy including weight format, KV cache format, and TP configuration. Calculate the resulting cost per 1M tokens.

Expected answer: Step 1 — Hardware and quality constraints: H100 has FP8 hardware (Transformer Engine). Quality target <1.0% MMLU. FP8 W8A8 gives -0.2% MMLU (well within target). AWQ INT4 gives -0.8% MMLU (within target). Choose FP8 W8A8 first since it's simpler and almost lossless. Step 2 — Model fit: FP8 LLaMA-3 70B = 70 GB → fits on single H100 (80 GB) with 10 GB headroom. No TP required. With 8 GPUs: 8 independent serving instances. Step 3 — KV cache: FP8 KV cache (0.67 GB/seq at S=4096). With 10 GB KV budget per GPU: 14 concurrent sequences. For TTFT <400ms: chunked prefill with chunk_size=512. Step 4 — Throughput: FP8 decode at H100: ~2,200 tok/s per GPU. 8 GPUs: 17,600 tok/s. Per day: 17,600 × 86,400 = 1.52B tokens/day. Needed: 50M/day. Fleet utilization: 50M / 1,520M = 3.3% — massively over-provisioned! Reduce to 1 serving instance for development; scale to 2 for production with headroom. Step 5 — Cost per 1M tokens: At 70% utilization on 1 GPU: effective tok/s = 2,200 × 0.70 = 1,540. Cost: $1.66/hr / (1,540 × 3,600) = $0.000299 = $0.299/1M tokens. If deployed on cloud (reserved H100): $2.50/hr / (1540 × 3600) = $0.451/1M tokens. Recommendation: FP8 W8A8 weights + FP8 KV cache, single H100 per serving instance, no TP needed. TP=2 only if P99 TTFT cannot be met with chunked prefill on single GPU.

Difficulty: Principal Level

Question 4 (AWQ vs GPTQ Trade-off and Production Eval): Your team runs GPTQ INT4 group=128 quantization on LLaMA-3 70B. Benchmark results: MMLU 79.0% (BF16: 80.3%), but your production use case is code completion. HumanEval shows: GPTQ 62.1% vs BF16 69.2%. This is a 10.2% relative drop. Prescribe the complete remediation strategy. Should you switch to FP8? To AWQ? Or accept the delta?

Expected answer: The 10.2% relative HumanEval drop is too large for code completion production use. This is a concrete example of why evaluating only on MMLU is insufficient — 1.3% MMLU delta masked a 7.1% HumanEval delta. Remediation in order of ease: (1) Try AWQ INT4: AWQ typically recovers 0.5% MMLU vs GPTQ, and the activation-aware scaling often helps code tasks more than knowledge tasks (code activations tend to have more concentrated, predictable outlier patterns). Run HumanEval on AWQ version and compare. Expected: AWQ may close gap to ~65-67% HumanEval (-3 to -6% vs BF16). (2) If AWQ insufficient: switch to FP8 W8A8. FP8 gives -0.2% MMLU and typically -0.3% HumanEval (near lossless). This eliminates the code quality concern entirely. Cost: doubles weight bytes vs INT4, but LLaMA-3 70B FP8 still fits on single H100. Decision: the 2× memory cost (35 GB → 70 GB) is acceptable if this is H100 with 80 GB. (3) If FP8 is not an option (e.g., A100): try GPTQ with group_size=64 (finer granularity, more scale factors, better quality, ~5% more storage). (4) Last resort: code-specific QAT fine-tuning with code-heavy calibration data. Most likely recommendation: switch to FP8 (zero calibration, best quality). The HumanEval gap disappears, model fits on H100, throughput increases vs BF16. INT4 is not appropriate for code generation tasks where precision of arithmetic step-by-step matters.

Further Reading

1. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers* — Frantar et al. (2022). `arxiv.org/abs/2210.17323` The foundational INT4 PTQ paper. Read the OBQ derivation and error propagation proof.
2. *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration* — Lin et al. (2023). `arxiv.org/abs/2306.00978` The salient weight insight is elegant. Read Sections 3-4 for the algorithm derivation.
3. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models* — Xiao et al. (2022). `arxiv.org/abs/2211.10438` The activation difficulty migration math. Best treatment of the outlier problem.
4. *QLoRA: Efficient Finetuning of Quantized LLMs* — Dettmers et al. (2023). `arxiv.org/abs/2305.14314` NF4 derivation, double quantization, paged Adam. Required reading for fine-tuning.
5. *NVIDIA Transformer Engine Documentation*. `docs.nvidia.com/deeplearning/transformer-engine/` The FP8 scaling algorithm and TE API. Read the "FP8 Formats" and "Training Recipe" sections before implementing FP8 training.
6. *FlexGen: High-throughput Generative Inference of Large Language Models with a Single GPU* — Sheng et al. (2023). `arxiv.org/abs/2305.05920` Treats quantization as part of a broader memory hierarchy strategy. Good systems view.

— End of Chapter 8 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved